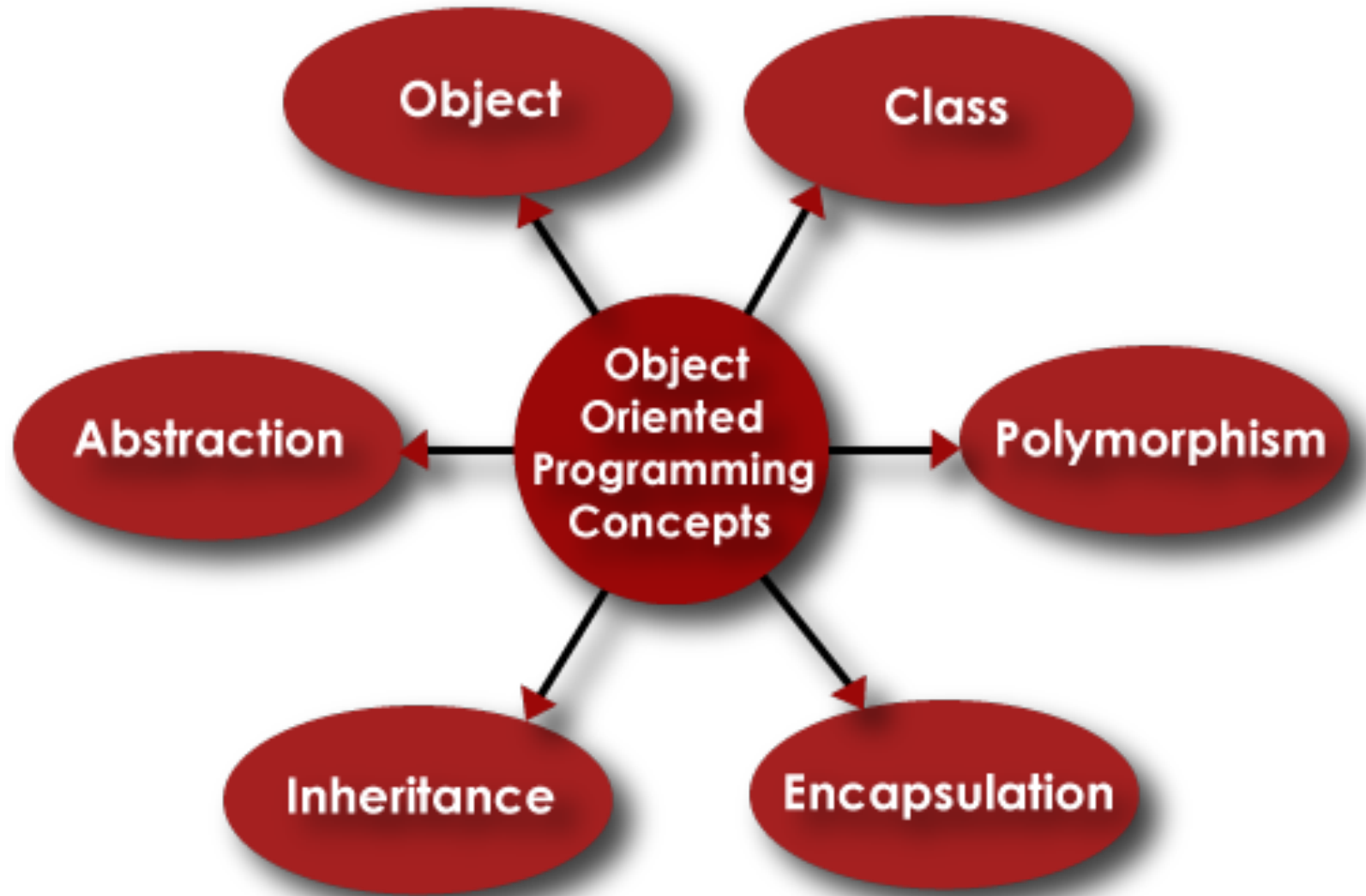


Object Oriented Programming (OOP)



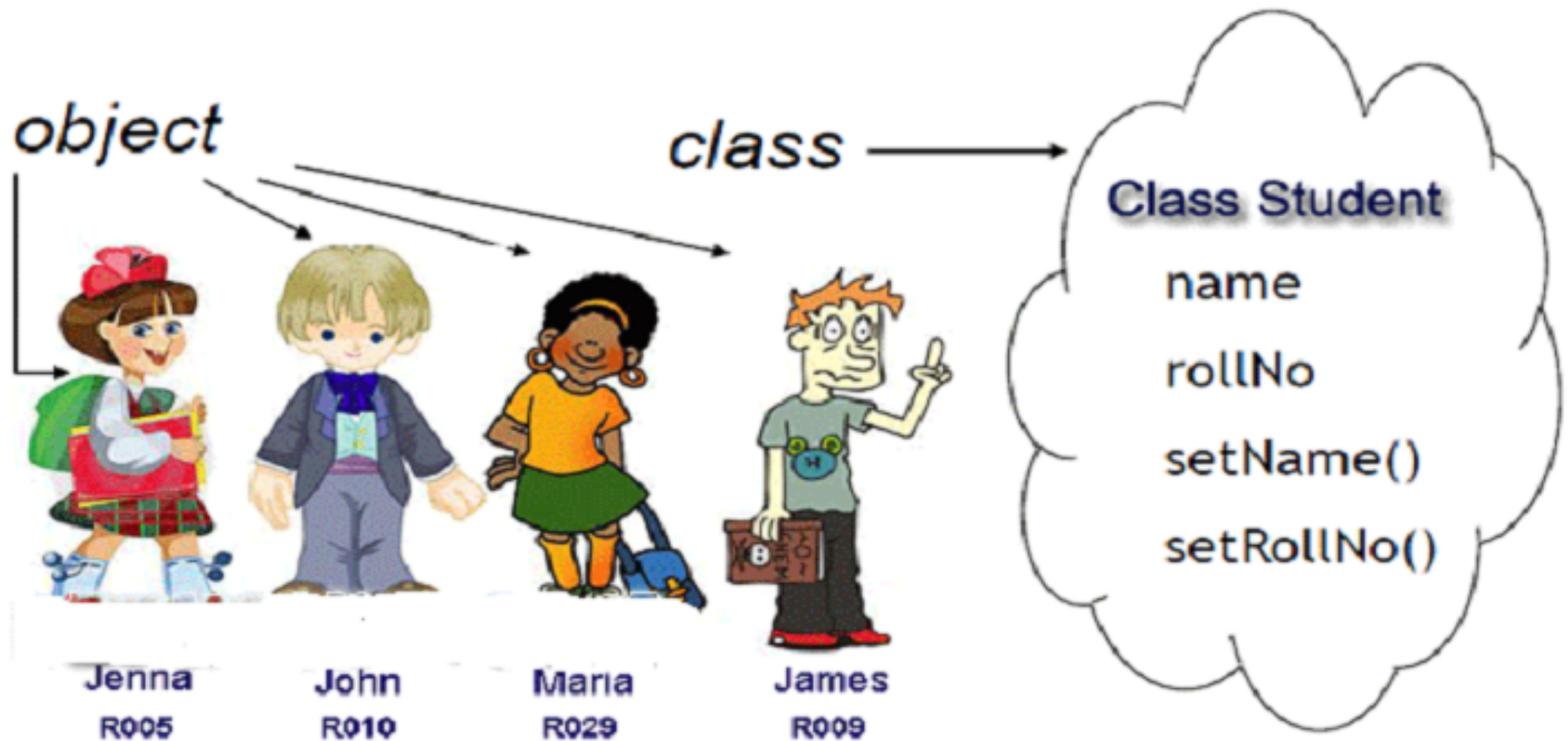
C++: What is a **Class**?

A **class** is a **blueprint** to create **objects**. It groups **data** (what it has) and **methods** (what it can do) into one new type.

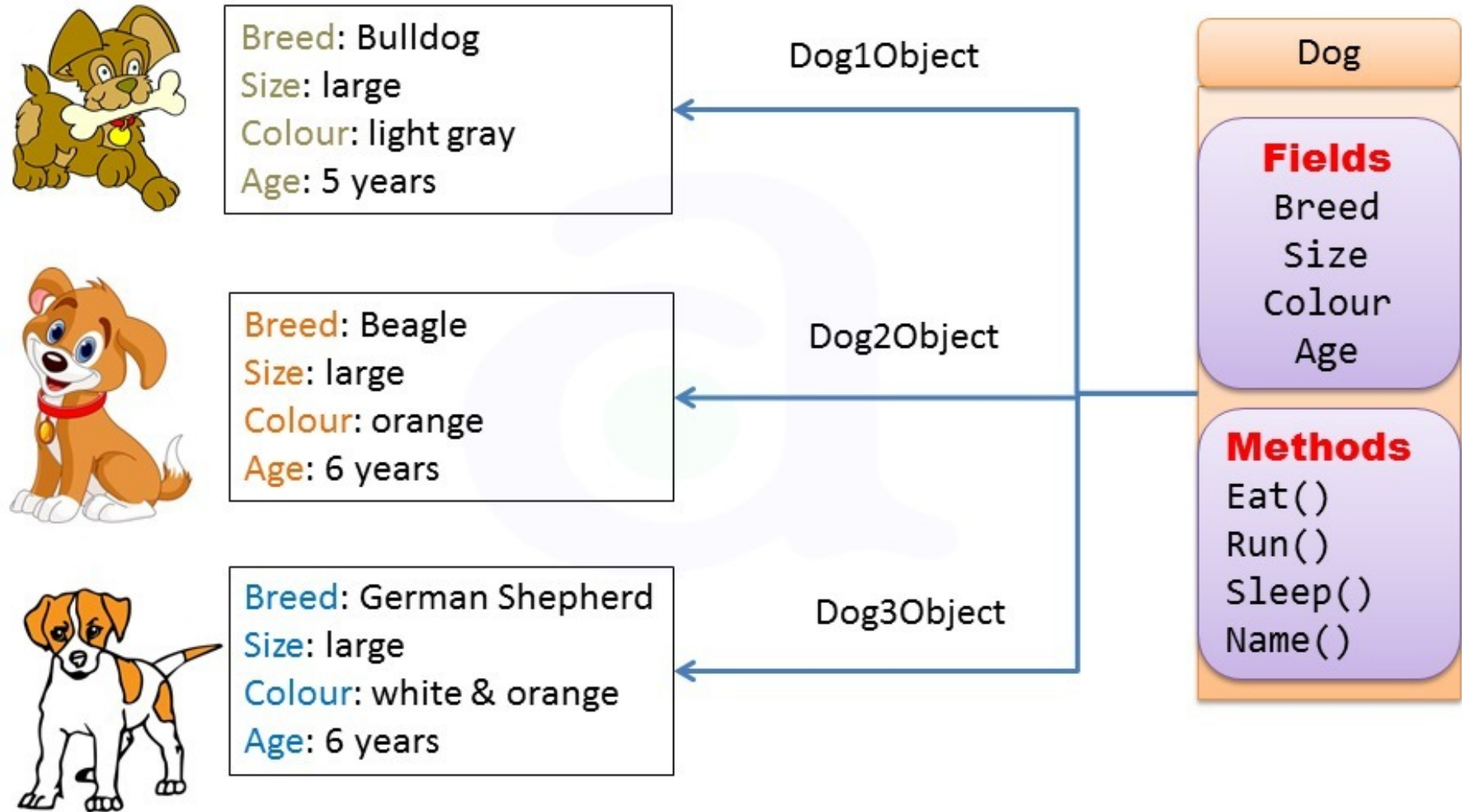
- **Data (fields):** variables stored inside each object (e.g., `balance`).
- **Methods:** functions that act on that data (e.g., `deposit()`, `withdraw()`).
- **Access:** `private`, `protected`, `public`.
- **Constructor:** special method with the same name of class; it *initializes* an object's data.
- **Object:** instance of class; one real thing made from the class or blueprint.

Takeaway: Class = your own type; object = a variable of that type; methods = actions you can perform on the object.

Class and Object



Class and Object



Example-1: Rectangle Class

Define a class to represent a **rectangle**. Include:

Data members

- Length
- Width

Member functions

- Assign initial values (read length and width)
- Compute area
- Compute perimeter
- Display length, width, area, perimeter

Example-1: Rectangle Class

```
1 // Only <iostream>, simple types
2 #include <iostream>
3 using namespace std;
4
5 class Rectangle {
6 private:
7     float length;
8     float width;
9
10 public:
11     // Constructor (Option 1: assign inside the body)
12     Rectangle() { length = 0.0f; width = 0.0f; }
13
14     // Assign initial values from user
15     void assignInitial() {
16         cout << "Enter length and width: ";
17         cin >> length >> width;
18     }
```

Example-1: Rectangle Class

```
1 // Compute area and perimeter
2 float area() { return length * width; }
3 float perimeter() { return 2 * (length + width); }
4
5 // Display all values
6 void display() return {
7     cout << "\n--- Rectangle ---\n";
8     cout << "Length      : " << length << "\n";
9     cout << "Width       : " << width << "\n";
10    cout << "Area        : " << area() << "\n";
11    cout << "Perimeter   : " << perimeter() << "\n";
12 }
13 }; // end class
```

Example-1: Rectangle Class

```
1 int main() {  
2     Rectangle r;           // create object  
3     r.assignInitial();     // read input  
4     r.display();          // show output  
5     return 0;  
6 }
```

Example-2: Account Class

Define a class to represent a **bank account**. Include:

Data members

- Name of depositor
- Account number
- Type of account
- Balance amount

Member functions

- Assign initial values
- Deposit an amount
- Withdraw an amount (after checking balance)
- Display name and balance

Example-2: Account Class

```
1 // Only <iostream>, beginner-friendly I/O
2 #include <iostream>
3 using namespace std;
4
5 class BankAccount {
6 private:
7     char    name[50];    // depositor name
8     int     accNo;       // account number
9     char    accType;     // 'S' = Savings, 'C' = Current
10    float   balance;     // balance amount
11
12 public:
13     // Constructor (Option 1: assign inside the body)
14     BankAccount () {
15         name[0] = '\0';
16         accNo = 0;
17         accType = 'S';
18         balance = 0.0f;
19     }
```

Example-2: Account Class

```
1  // Deposit
2  void deposit(float amount) {
3      if (amount > 0.0f) balance += amount;
4      else cout << Invalid deposit amount.\n;
5  }
6
7  // Withdraw (with check)
8  void withdraw(float amount) {
9      if (amount <= 0.0f) {
10         cout << Invalid withdrawal amount.\n;
11     } else if (amount > balance) {
12         cout << Insufficient funds. Available:  <<
balance << \n;
13     } else {
14         balance -= amount;
15     }
16 }
17
18 // Display
```

Example-2: Account Class

```
1 int main() {  
2     BankAccount acc;  
3  
4  
5     acc.assignInitial();  
6  
7     float dep;  
8     cout << "\nAmount to deposit: ";  
9     cin >> dep;  
10    acc.deposit(dep);  
11  
12    float wd;  
13    cout << "Amount to withdraw: ";  
14    cin >> wd;  
15    acc.withdraw(wd);  
16  
17    acc.display();  
18    return 0;  
19 }
```

Constructor

- **What is a constructor?**
 - A constructor is a **special member function** that **initializes an object** at the moment it is created.
 - **What are the properties of a constructor?**
- ① A constructor is a **special type of function**, with or without parameter.
 - ② It has the **same name as the class**.
 - ③ It has **no return type** (not even `void`).
 - ④ It is **called automatically** when an object is created.
 - ⑤ It is used to initialize the member variable or data.
 - ⑥ Constructor is optional in the class.

Default Constructor (without Parameter)

```
1 // Tiny example: default (no-parameter) constructor
2 #include <iostream>
3 using namespace std;
4
5 class Counter {
6 private:
7     int value;
8 public:
9     Counter() {                // default ctor
10         value=0;
11     }
12
13     void increment() { value = value + 1; }
14     int  get() { return value; }
15 };
16
17 int main() {
18     Counter obj;                // default constructor runs here
19     obj.increment();
20     cout << "Now value = " << obj.get() << "\n";
21 }
```

Constructor (with Parameter)

```
1 // Tiny example: default (no-parameter) constructor
2 #include <iostream>
3 using namespace std;
4
5 class Counter {
6 private:
7     int value;
8 public:
9     Counter(float start) {                // default ctor
10         value=start;
11     }
12
13     void increment() { value = value + 1; }
14     int  get() { return value; }
15 };
16
17 int main() {
18     Counter obj1(5), obj2(10), obj3(15);
19     obj1.increment();
20     cout << "Now value = " << obj1.get() << "\n";
21 }
```

Memory Allocation for Multiple Objects

